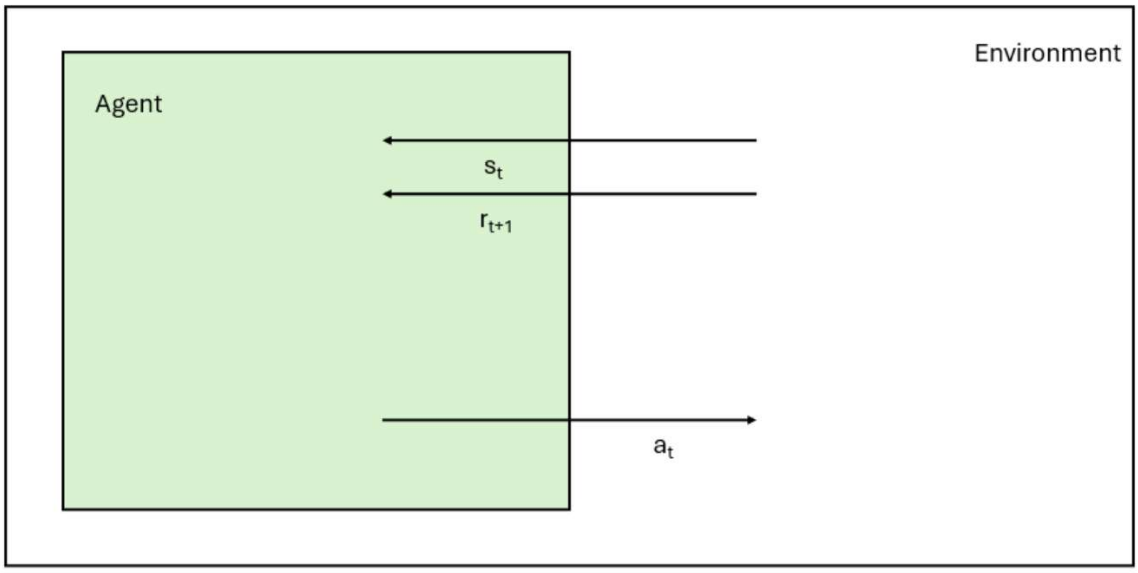
Deep Reinforcement Learning

March 23, 2026 8:33 AM



t - time

s_t - state of environment at time t

a_t - action taken at time t

r_{t+1} - reward provided by environment at time t.
$r(s_t, a_t)$

γ - discount factor, indicates how much we value long-term rewards

$\pi(s)$ - policy to take actions given state s

$V(s)$ - a value function that estimates long-term rewards associated with s

$Q(s, a)$ - an action-value function that estimates long-term rewards associated with s, a

$Q(s, a)$ - an action-value

long-term rewards associated with s, a

$$V^{\pi}(s_t) = E\left(\sum_{i=0}^{\infty} \gamma^i r_{t+i}\right)$$

$$Q^{\pi}(s_t, a_t) = E\left(\sum_{i=0}^{\infty} \gamma^i r_{t+i}\right)$$

Under policy π:

$$V^{\pi}(s) = Q^{\pi}(s_t, \pi(s_t))$$

Optimal policy - the policy that leads to biggest long-term reward

Call V^* and Q^* the value, action-value functions under optimal policy.

$$V^*(s_t) = \max_a Q^*(s_t, a)$$

Write Q recursively

$$Q^{\pi}(s_t, a_t) = E\left(\sum_{i=0}^{\infty} \gamma^i r_{t+i}\right)$$

$$= E(r_t) + E\left(\sum_{i=1}^{\infty} \gamma^i r_{t+i}\right)$$

$$= r_t + E\left(\sum_{j=0}^{\infty} \gamma^{j+1} r_{t+j+1}\right)$$

$$= r_t + \gamma E\left(\sum_{j=0}^{\infty} \gamma^j r_{t+1+j}\right)$$

$$= r_t + \gamma \sum_{s'} P(s' | s_t, a_t) Q^{\pi}(s', \pi(s'))$$

$$Q^{\pi}(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) Q^{\pi}(s', \pi(s'))$$

Bellman Equation

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s', a')$$

Bellman Equation for optimal policy

Optimal policy

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

Optimal Policy

$$\pi^*(s) = \underset{a}{\operatorname{argmax}}\, Q^*(s,a)$$

Option 1. Adaptive Dynamic Programming

1. Have an agent interact with environment under random policy.
2. Use these to compute $P(s' \mid s,a)$.
3. Initialize $Q(s,a)$ (often $Q(s,a) = r(s,a)$).
4. Iterate on Bellman Equation to compute $Q^*(s,a)$.

Issue:

Rarely sample some state-action pairs

Computationally expensive.

Requires states · actions to have a small number of discrete possible values.

Deep Q-Learning

Similar state-action pairs have similar values for Q.

Represent Q as a parameterized function of state, action.

state, action.

$$Q(s, a; \theta)$$

Learn parameters to make the two sides of Bellman Equation as close as possible.

Benefits:

Handles continuous state-action pairs.

Q updated for all state-action pairs

Let Q be a neural network.

We want to use:

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial L}{\partial \theta_i}$$

Define loss function

$$L = \frac{1}{2}\left(Q(s_t, a_t; \theta) - Y\right)^2$$

$$Y = r(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a'; \theta)$$

"how good our approximation of Q is"

now good our appr

Differentiate L...

$$\frac{\partial L}{\partial \theta_i} = \left(Q(s_t, a_t; \theta) - Y\right) \cdot \frac{\partial}{\partial \theta_i} Q(s_t, a_t; \theta)$$

Assume Y is independent of θ.
(slightly incorrect).

Algorithm For Deep Q-Learning

0. Randomly initialize parameters θ.
1. Given the current state s_t. take action a_t with a trade-off between exploration and exploitation. Leads to s_{t+1}.
2. Forward propagate s_t, a_t through Q.
   Forward propagate s_{t+1}, with all possible a' through Q.
3. Compute L and $\partial L/\partial \theta_i$ for all θ_i.
   Update θ_i using gradient descent.
4. Repeat until convergence.

A few practical strategies:

A few practical strategies:

1. Architecture $Q = Q(s_t; \theta)$ and produce a Q-value for all possible actions.
   Number of neurons in output layer
   = Number of possible actions.

2. Only update Q in Y every C iterations
   Tends to lead to better convergence.

3. Do batch updates to θ by taking subset of recent (s_t, a_t, s_{t+1}) tuples.